



中国海洋大学
OCEAN UNIVERSITY OF CHINA

系统开发工具基础实验报告

第 3 周: Packaging and Shipping Code; 智能体编程; 不止于代码;
Python 与 PyTorch

姓名: 姬艺华

学号: 25020007046

日期: 2026 年 9 月 7 日

目录

1	课上习题	3
1.1	第 9 题：从源码构建并在干净环境安装 Wheel	3
1.1.1	任务说明	3
1.1.2	关键操作	3
1.1.3	实验结果	4
1.2	第 10 题：让编程智能体进入可验证的修复循环	4
1.2.1	任务说明	4
1.2.2	关键操作	4
1.2.3	实验结果	5
1.3	第 11 题：用调试器定位归并排序缺陷	6
1.3.1	任务说明	6
1.3.2	关键操作	6
1.3.3	实验结果	7
1.4	第 12 题：修复一个可复现的线性回归训练循环	8
1.4.1	任务说明	9
1.4.2	关键操作	9
1.4.3	实验结果	10
2	课下习题	10
2.1	查看 Python 环境信息	10
2.2	创建 Python 包并本地安装	11
2.3	虚拟环境隔离测试	12
2.4	pytest 测试函数	13
2.5	pytest 发现并修复错误	13
2.6	Shell 自动运行 Python 程序	14
2.7	Python 日志记录	15
2.8	增加错误日志	15
2.9	使用 requirements.txt 管理项目依赖	16
2.10	PyTorch 模型保存与加载	17
3	Git 与作业提交	18
3.1	仓库内容整理	18
3.2	GitHub / Gitee 提交记录	19
3.3	commit 记录截图、仓库页面截图	19

4 问题与总结	19
4.1 遇到的问题	19
4.2 实验总结	20

1 课上习题

本模块记录上课实验四道例题操作基本过程，以及关键点。

1.1 第 9 题：从源码构建并在干净环境安装 Wheel

主题：Packaging and Shipping Code

1.1.1 任务说明

创建一个简单的 Python 命令行工具，并将源码打包生成 Wheel 安装包；随后在新的虚拟环境中安装生成的 Wheel 文件，验证安装后的命令是否可以正常运行。

1.1.2 关键操作

创建 `src/greetlab/_init_.py`、给定的 `cli.py` 和 `pyproject.toml`；在已安装 build 工具的课程环境中运行 `python -m build`，生成 wheel；新建一个干净虚拟环境，只从生成的 wheel 安装；切换到 q09 之外运行 `sdt-greet --name:`

```
pip install build
python -m build
python -m venv clean_env
clean_env\Scripts\activate
pip install dist\greetlab_25020007046-0.1.0-py3-none-any.whl
sdt-greet --name 姬艺华
```

关键点：

- 使用标准 Python 项目结构，将源代码放入 `src` 目录中，`greetlab` 作为 Python 包。
- 配置项目打包信息：`pyproject.toml` 用于描述 Python 项目的构建方式和元数据。其中 `sdt-greet = "greetlab.cli:main"` 表示安装该包后，可以通过 `sdt-greet` 命令调用 `cli.py` 中的 `main()` 函数。
- `pip install build` 安装 Python 官方构建工具，用于将源码转换为可安装的 Wheel 文件。
- `python -m build` 生成 `dist` 文件，`build` 根据 `pyproject.toml` 配置，将源码打包。其中 `.whl`：Python 二进制安装包，用于安装；`.tar.gz`：源码分发包。
- `python -m venv clean_env` 创建独立 Python 环境，模拟其他用户安装该包的情况，避免受到系统环境影响。
- `pip install dist/greetlab_25020007046-0.1.0-py3-none-any.whl` 从生成的 Wheel 文件安装 Python 包。

1.1.3 实验结果

```
Successfully built greetlab_25020007046-0.1.0.tar.gz and greetlab_25020007046-0.1.0-py3-none-any.whl
PS D:\classPai\Class\8.24\q09> python -m venv clean_env
PS D:\classPai\Class\8.24\q09> clean_env\Scripts\activate
(clean_env) PS D:\classPai\Class\8.24\q09> pip install dist\greetlab_25020007046-0.1.0-py3-none-any.whl
Processing d:\classpai\class\8.24\q09\dist\greetlab_25020007046-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007046
Successfully installed greetlab-25020007046-0.1.0
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'D:\classPai\Class\8.24\q09\clean_env\Scripts\python.exe -m pip install --u
pgrade pip' command.
(clean_env) PS D:\classPai\Class\8.24\q09> sdt-greet --name 姬艺华
Hello, 姬艺华!
(clean_env) PS D:\classPai\Class\8.24\q09> |
```

图 1 确认运行结果

1.2 第 10 题：让编程智能体进入可验证的修复循环

主题：智能体编程

1.2.1 任务说明

本实验要求在第 9 题 Python 命令行工具的基础上，引入自动化测试流程。通过添加测试用例发现程序缺陷，利用编程智能体分析问题并修改代码，最后通过测试验证修复结果。

1.2.2 关键操作

创建 q10 项目（复制 q09），编写测试文件夹 tests 以及测试文件 test_cli.py，并运行，发现首次运行失败：

```
1 #tests/test_cli.py
2 import pytest
3 from greetlab.cli import main
4
5 def test_empty_name(monkeypatch):
6     monkeypatch.setattr(
7         "sys.argv",
8         ["sdt-greet", "--name", " "]
9     )
10
11     with pytest.raises(SystemExit):
12         main()
```

其中 `with pytest.raises(SystemExit): main()` 表示测试要求当输入只有空白字符时，程序应该异常退出。如果程序没有退出，则测试失败。

pytest

Failed: DID NOT RAISE <class 'SystemExit'>

表示当前程序没有处理空白参数，需要进行代码修复。

修改程序增加输入检查，使用 `strip()` 删除字符串两端空格，当输入为空白字符时退出程序，

避免无效输入:

```

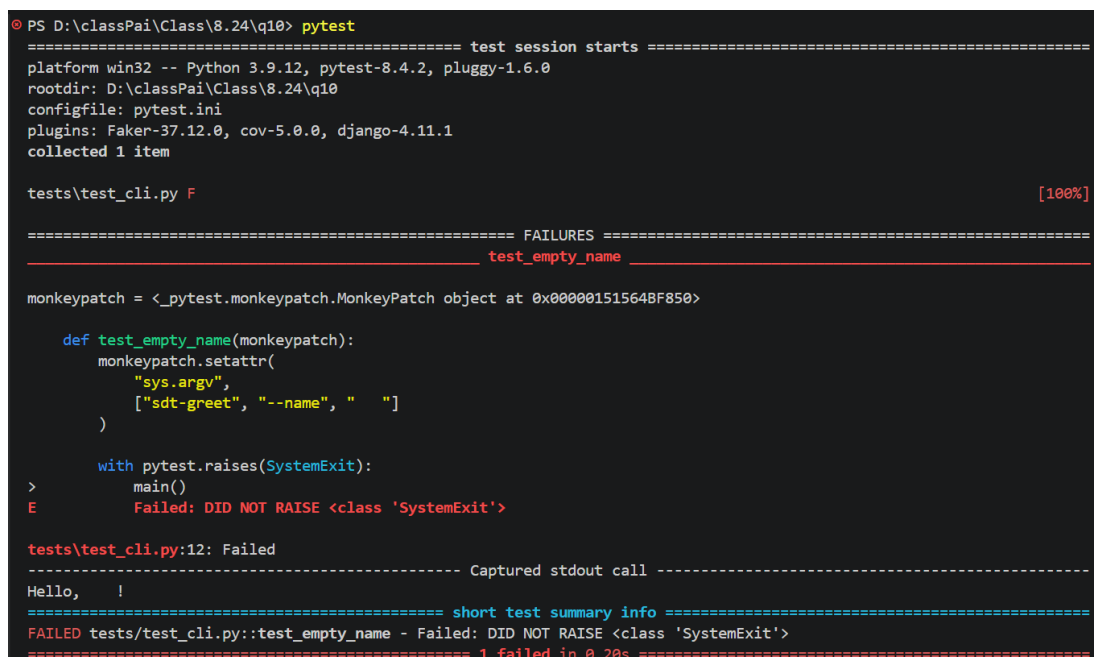
1 import argparse
2
3 def main():
4     p = argparse.ArgumentParser()
5     p.add_argument("--name", required=True)
6     a = p.parse_args()
7
8     if not a.name.strip():
9         raise SystemExit(2)
10
11     print(f"Hello, {a.name}!")
12
13 if __name__ == "__main__":
14     main()

```

关键点:

- 测试驱动修复流程: 发现问题-> 编写测试-> 运行失败-> 修改代码-> 测试通过
- pytest 可以自动执行测试用例, 帮助快速发现程序错误。
- SystemExit 用于让命令行程序返回错误状态。
- strip() 可以去除字符串中的空格, 用于判断用户输入是否有效。
- 自动化测试可以降低人工检查成本, 提高代码可靠性。

1.2.3 实验结果



```

PS D:\classPai\Class\8.24\q10> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q10
configfile: pytest.ini
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

tests\test_cli.py F [100%]

===== FAILURES =====
_____ test_empty_name _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x000001515648F850>

    def test_empty_name(monkeypatch):
        monkeypatch.setattr(
            "sys.argv",
            ["sdt-greet", "--name", " "]
        )

        with pytest.raises(SystemExit):
            > main()
E         Failed: DID NOT RAISE <class 'SystemExit'>

tests\test_cli.py:12: Failed
----- Captured stdout call -----
Hello, !
===== short test summary info =====
FAILED tests/test_cli.py::test_empty_name - Failed: DID NOT RAISE <class 'SystemExit'>
===== 1 failed in 0.20s =====

```

图 2 修复前测试失败结果

```
PS D:\classPai\Class\8.24\q10> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q10
configfile: pytest.ini
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

tests\test_cli.py . [100%]

===== 1 passed in 0.11s =====
PS D:\classPai\Class\8.24\q10>
```

图 3 修复后测试通过结果

1.3 第 11 题：用调试器定位归并排序缺陷

主题：不止于代码

1.3.1 任务说明

本实验要求将一个模糊的软件问题描述整理为规范的协作记录，包括 Issue、提交信息和代码评审意见。通过 Markdown 文件记录问题背景、修复方案以及 Review 结果，模拟真实软件开发中的协作流程。

1.3.2 关键操作

创建 communication.md。

使用 Markdown 文件记录软件开发过程中的问题反馈、修改说明和评审意见，方便团队成员理解和追踪问题。

编写 issue 问题描述：问题标题，运行环境，问题描述，复现步骤，期望结果，实际结果。

将模糊的问题描述转换为结构化信息，使开发人员能够快速理解问题并进行修复。

关键点：

- Issue 需要描述清楚问题背景、复现方法以及期望行为。
- 提交信息应该简洁明确，能够反映修改目的。
- Code Review 不仅检查代码是否正确，还需要关注潜在风险和优化方向。
- Markdown 是软件开发中常用的文档格式，可以方便地在 GitHub 等平台展示和维护。

1.3.3 实验结果

sdt-greet 空姓名输入问题修复记录

Issue

标题

修复 sdt-greet 对空白姓名参数校验不足的问题

环境

操作系统: Windows

运行环境: Python 3.9

问题描述

sdt-greet 命令行工具目前未对 name 参数进行有效性检查。

当用户输入只包含空格的姓名时:

```
sdt-greet --name " "
```

程序仍然输出:

```
Hello, !
```

并返回退出状态码 0。

该行为导致无效参数被接受, 不符合预期。

复现步骤

1. 安装并运行 sdt-greet。
2. 执行命令:

```
sdt-greet --name " "
```

图 4 md 文件内容



重现步骤

- 1. 安装并运行 sdt-greet。
- 2. 执行命令：

```
sdt-greet --name " "
```

- 3. 查看终端输出和退出状态。

期望结果

当 name 参数为空或仅包含空格时，程序应提示输入错误，并返回非正常退出状态。

实际结果

程序正常执行，并输出无效的问候信息。

未知信息

导致该问题的具体原因待确认。

提交信息

修复空姓名参数校验问题

在命令行参数解析后增加 name 参数检查。
当检测到输入内容为空或仅包含空白字符时，程序主动退出并返回错误状态，避免生成无效输出。
同时保留正常姓名输入时的原有功能。

图 5 md 文件内容

评审意见

Review Result

通过。

Comments

Blocking:
无阻塞问题，可以合并。
Suggestion:
建议增加空字符串和全空格字符串的自动化测试，防止后续修改导致该问题重新出现。
Nit:
错误提示信息可以进一步优化，例如明确提示用户重新输入有效姓名。
本次修改解决了空姓名输入问题，修改范围合理，没有影响其他正常功能。

图 6 md 文件内容

1.4 第 12 题：修复一个可复现的线性回归训练循环

主题：Python 与 PyTorch

1.4.1 任务说明

本实验使用 PyTorch 完成一个简单线性回归训练任务，通过梯度下降优化模型参数，使模型学习目标函数 $y=3x-1$ 。

1.4.2 关键操作

补全训练循环，调用 `zero_grad`、`backward`、`step`；训练后用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失；打印最终损失 `weight` `bias`：

```
1 import torch
2 from torch import nn
3 torch.manual_seed(20260907)
4 x = torch.linspace(-1, 1, 101).reshape(-1, 1)
5 y = 3 * x - 1
6 model = nn.Linear(1, 1)
7 loss_fn = nn.MSELoss()
8 opt = torch.optim.SGD(model.parameters(), lr=0.1)
9 for _ in range(200):
10     pred = model(x)
11     loss = loss_fn(pred, y)
12     opt.zero_grad()
13     loss.backward()
14     opt.step()
15 model.eval()
16 with torch.no_grad():
17     pred = model(x)
18     final_loss = loss_fn(pred, y)
19     print("loss:", final_loss.item())
20     print("weight:", model.weight.item())
21     print("bias:", model.bias.item())
```

代码解释：

- `model = nn.Linear(1, 1)` 创建线性模型，数学形式 $y = wx + b$ ，`w`、`b` 不知道，需要训练。
- `loss_fn = nn.MSELoss()` 定义损失函数，计算预测值和真实值之间差多少。
- `opt = torch.optim.SGD(model.parameters(), lr=0.1)` 优化器，SGD 随机阶梯下降；作用：根据错误方向调整 `weight`、`bias`。

三个 TODO：

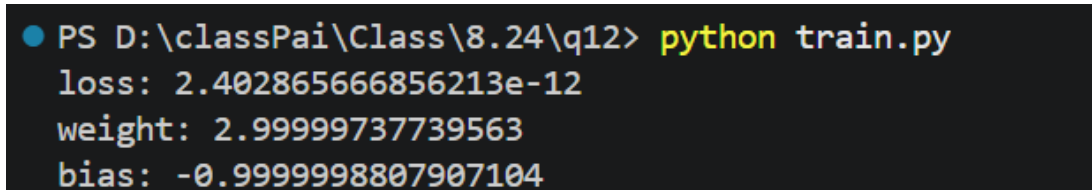
- 清空梯度：`opt.zero_grad()`，每轮训练开始要清空。
- 反向传播：`loss.backward()`，损失函数对于参数的梯度。
- 更新参数：`opt.step()`，根据梯度修改 `weight`、`bias`。
- 进入评估模式：`model.eval()`，表示模型现在不是训练状态。
- 不计算梯度：`with torch.no_grad()`：只是查看结果，不训练。

- 计算最终结果: `with torch.no_grad():pred = model(x) final_loss = loss_fn(pred, y) print("loss:", final_loss.item()) print("weight:", model.weight.item()) print("bias:", model.bias.item())`

关键点:

- PyTorch 训练流程通常包括: 前向传播、计算损失、反向传播、参数更新。
- 梯度是模型优化的重要依据。
- 训练阶段和评估阶段需要区分。
- 不能直接给 `weight` 和 `bias` 赋值, 需要通过优化过程学习参数。

1.4.3 实验结果



```
● PS D:\classPai\Class\8.24\q12> python train.py
loss: 2.40286566856213e-12
weight: 2.99999737739563
bias: -0.999998807907104
```

图 7 训练后内容

2 课下习题

2.1 查看 Python 环境信息

查看当前 Python 版本和安装的第三方包。

```
python --version
python -m pip list
```

了解当前开发环境, 方便排查依赖问题。

```
● PS D:\classPai\Class\8.24\test> python --version
Python 3.9.12
● PS D:\classPai\Class\8.24\test> python -m pip list
Package                                Version
-----
aliyun-python-sdk-core                 2.16.0
aliyun-python-sdk-kms                 2.16.5
amqp                                    5.3.1
asgiref                                3.11.1
async-timeout                          5.0.1
attrs                                  25.4.0
billiard                               4.2.4
build                                  1.4.4
celery                                  5.3.6
certifi                                2026.2.25
cffi                                    2.0.0
charset-normalizer                     3.4.5
click                                  8.1.8
click-didyoumean                       0.3.1
click-plugins                          1.1.1.2
click-repl                             0.3.0
colorama                               0.4.6
contourpy                              1.3.0
coverage                               7.10.7
crcmod                                 1.7
cryptography                           46.0.5
cyclor                                 0.12.1
```

图 8 终端输出

2.2 创建 Python 包并本地安装

创建一个简单 Python 包，并使用 pip 安装，然后测试是否可以调用。

hello.py

```
1 def say():
2     return "hello system tools"
```

pyproject.toml

```
1 [build-system]
2 requires=["setuptools"]
3 build-backend="setuptools.build_meta"
4 [project]
5 name="hello-package"
6 version="0.1.0"
```

安装、测试、运行：

```
pip install .
```

test.py

```
1 from hello.hello import say
2
3 print(say())
```

python test.py

```
PS D:\classPai\Class\8.24\test\test3> pip install .
Processing d:\classpai\class\8.24\test\test3
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: hello-package
Building wheel for hello-package (pyproject.toml) ... done
Created wheel for hello-package: filename=hello_package-0.1.0-py3-none-any.whl size=1337 sha256=4a8aa9c1a7151d7d1ea0bf995
3f4a6cc9390b0f242130427755da98467e6a9da
Stored in directory: C:\Users\lx\AppData\Local\Temp\pip-ephem-wheel-cache-2li25hyh\wheels\02\F4\01\9a90a85cbc263c43a476e5
45c6136c034ea733aa7f7fbad35a
Successfully built hello-package
Installing collected packages: hello-package
Successfully installed hello-package-0.1.0
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'C:\Users\lx\AppData\Local\Programs\Python\Python39\python.exe -m pip install --upgra
de pip' command.
PS D:\classPai\Class\8.24\test\test3> python test.py
hello system tools
```

图 9 查看输出结果

2.3 虚拟环境隔离测试

创建两个环境，观察 Python 包隔离。

```
python -m venv env1 创建环境
env1\Scripts\activate 激活环境
pip install requests 安装
pip list 查看
deactivate 退出
```

```
PS D:\classPai\Class\8.24\test> python -m venv test_env
PS D:\classPai\Class\8.24\test> test_env\Scripts\activate
```

图 10 查看

```

(test_env) PS D:\classPai\Class\8.24\test> pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Collecting idna<4,>=2.5
  Downloading idna-3.19-py3-none-any.whl (68 kB)
    68.5/68.5 KB 71.8 kB/s eta 0:00:00
Collecting charset_normalizer<4,>=2
  Downloading charset_normalizer-3.5.1-cp39-cp39-win_amd64.whl (206 kB)
    206.4/206.4 KB 10.4 kB/s eta 0:00:00
Collecting urllib3<3,>=1.21.1
  Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Collecting certifi>=2017.4.17
  Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
    137.0/137.0 KB 8.0 kB/s eta 0:00:00
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.1 idna-3.19 requests-2.32.5 urllib3-2.6.3
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'D:\classPai\Class\8.24\test\test_env\Scripts\python.exe -m pip install --upgrade pip' command.
(test_env) PS D:\classPai\Class\8.24\test> pip list
Package Version
-----
certifi 2026.7.22
charset-normalizer 3.5.1
idna 3.19
pip 22.0.4
requests 2.32.5
setuptools 58.1.0
urllib3 2.6.3
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'D:\classPai\Class\8.24\test\test_env\Scripts\python.exe -m pip install --upgrade pip' command.

```

图 11 查看

2.4 pytest 测试函数

使用 pytest 验证代码正确性。

```

pip install pytest 安装
math.py , test_math.py 创建
pytest 测试

```

```

1 #math.py
2 def add(a,b):
3     return a+b
4
5 #test_math.py
6 from math import add
7 def test_add():
8     assert add(1,2)==3

```

```

PS D:\classPai\Class\8.24\test\test4> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\test\test4
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

test_math.py . [100%]

===== 1 passed in 0.13s =====

```

图 12 查看

2.5 pytest 发现并修复错误

模拟智能编程修复过程。

```

1 #mymath.py
2 def multiply(a,b):

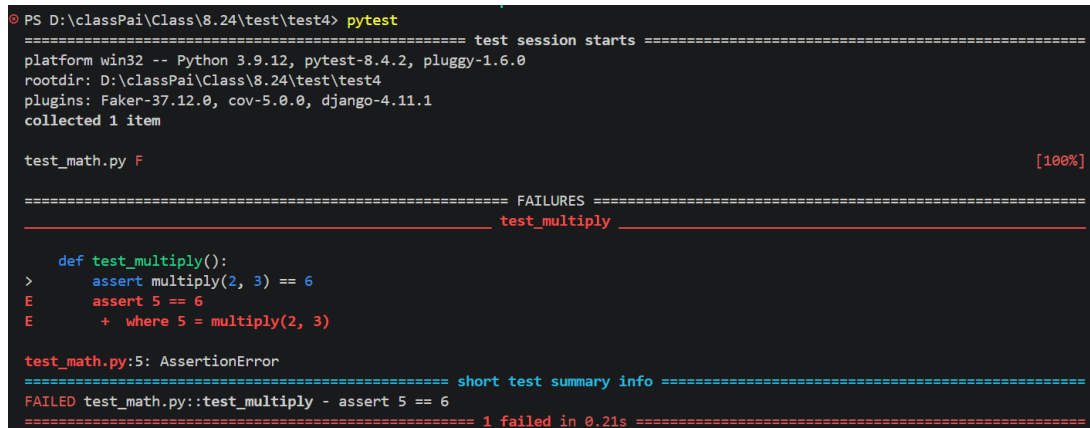
```

```

3     return a+b
4 #test_math.py
5 def test():
6     assert multiply(2,3)==6

```

修复只需要将 mytest 中的 a+b 改为 a*b。



```

PS D:\classPai\Class\8.24\test\test4> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\test\test4
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

test_math.py F [100%]

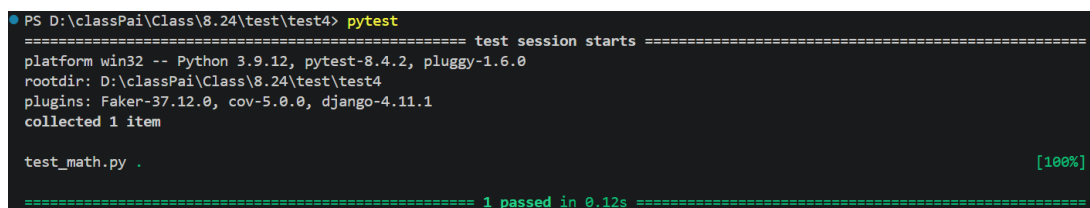
===== FAILURES =====
test_multiply

  def test_multiply():
>     assert multiply(2, 3) == 6
E       assert 5 == 6
E       + where 5 = multiply(2, 3)

test_math.py:5: AssertionError
===== short test summary info =====
FAILED test_math.py::test_multiply - assert 5 == 6
===== 1 failed in 0.21s =====

```

图 13 查看报错



```

PS D:\classPai\Class\8.24\test\test4> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\test\test4
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

test_math.py . [100%]

===== 1 passed in 0.12s =====

```

图 14 查看修复

2.6 Shell 自动运行 Python 程序

使用 Shell 脚本自动执行 Python 程序。

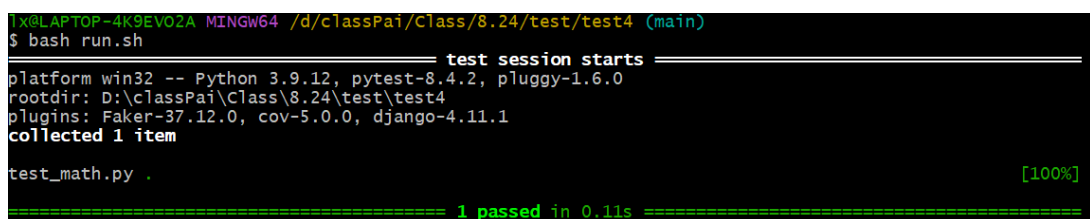
```

1 #run.sh
2 #!/bin/bash
3 pytest

```

运行:

```
bash run.sh
```



```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/test/test4 (main)
$ bash run.sh
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\test\test4
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 1 item

test_math.py . [100%]

===== 1 passed in 0.11s =====

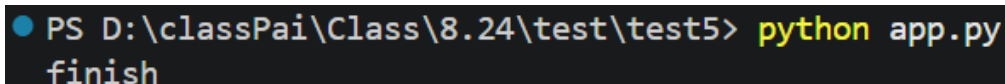
```

图 15 查看自动运行结果

2.7 Python 日志记录

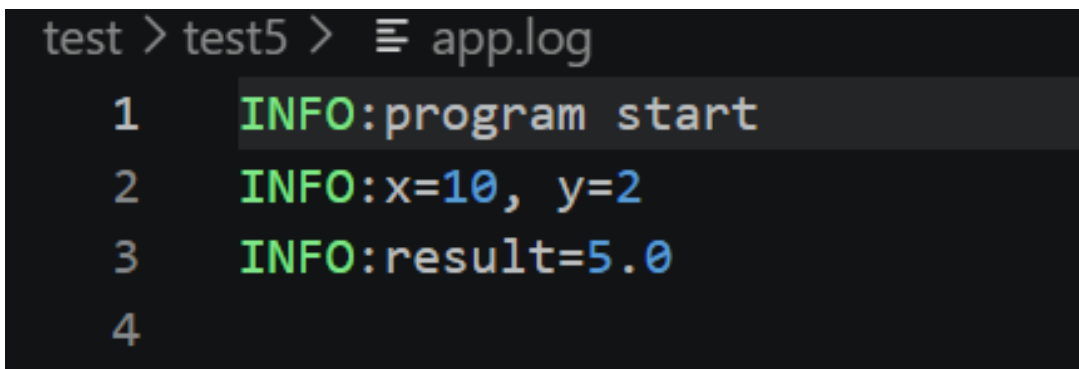
使用 logging 替代 print。

```
1 #app.py
2 import logging
3 logging.basicConfig(
4     filename="app.log",
5     level=logging.INFO,
6     format="%(levelname)s:%(message)s"
7 )
8 logging.info("program start")
9 x = 10
10 y = 2
11 logging.info(f"x={x}, y={y}")
12 result = x / y
13 logging.info(f"result={result}")
14 print("finish")
15 logging.info("program start")
```



```
PS D:\classPai\Class\8.24\test\test5> python app.py
finish
```

图 16



```
test > test5 > cat app.log
1 INFO:program start
2 INFO:x=10, y=2
3 INFO:result=5.0
4
```

图 17 查看输出日志

2.8 增加错误日志

```
1 #app.py
2 import logging
3 logging.basicConfig(
4     filename="app.log",
5     level=logging.INFO,
6     format="%(levelname)s:%(message)s"
7 )
8 logging.info("program start")
```

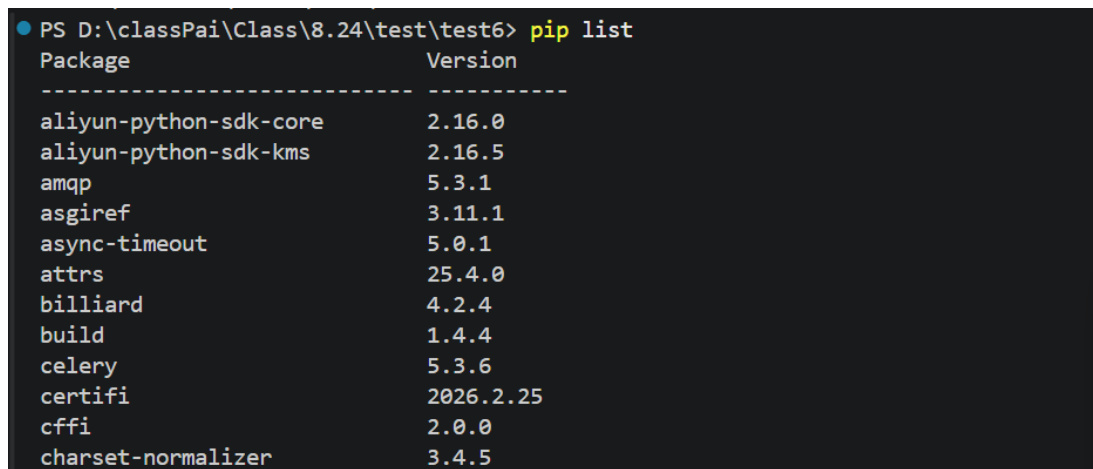
```
9 x = 10
10 y = 2
11 logging.info(f"x={x}, y={y}")
12
13 try:
14     result = x / y
15     logging.info(f"result={result}")
16 except Exception as e:
17     logging.error(e)
18
19 logging.info(f"result={result}")
20 print("finish")
```

此时如果 $y=0$ ，日志会记录错误。

2.9 使用 requirements.txt 管理项目依赖

在 Python 项目中，使用 requirements.txt 保存项目依赖，使其他环境可以快速恢复运行环境。如果其他人拿到项目可以直接：

```
pip install -r requirements.txt
```



```
PS D:\classPai\Class\8.24\test\test6> pip list
Package                                Version
-----
aliyun-python-sdk-core                 2.16.0
aliyun-python-sdk-kms                 2.16.5
amqp                                    5.3.1
asgiref                                3.11.1
async-timeout                          5.0.1
attrs                                  25.4.0
billiard                               4.2.4
build                                  1.4.4
celery                                 5.3.6
certifi                                2026.2.25
cffi                                    2.0.0
charset-normalizer                     3.4.5
```

图 18 查看电脑环境

```
PS D:\classPai\Class\8.24\test\test6> pip freeze > requirements.txt
PS D:\classPai\Class\8.24\test\test6> cat requirements.txt
aliyun-python-sdk-core==2.16.0
aliyun-python-sdk-kms==2.16.5
amqp==5.3.1
asgiref==3.11.1
async-timeout==5.0.1
attrs==25.4.0
billiard==4.2.4
build==1.4.4
celery==5.3.6
certifi==2026.2.25
cffi==2.0.0
charset-normalizer==3.4.5
click==8.1.8
click-didyoumean==0.3.1
click-plugins==1.1.1.2
```

图 19 查看文件

2.10 PyTorch 模型保存与加载

保存训练好的模型，并重新加载验证结果。

```
1 import torch
2 from torch import nn
3 torch.manual_seed(20260907)
4 x = torch.linspace(-1, 1, 101).reshape(-1, 1)
5 y = 3 * x - 1
6 model = nn.Linear(1, 1)
7 loss_fn = nn.MSELoss()
8 opt = torch.optim.SGD(model.parameters(), lr=0.1)
9 for _ in range(200):
10     pred = model(x)
11     loss = loss_fn(pred, y)
12     opt.zero_grad()
13     loss.backward()
14     opt.step()
15 model.eval()
16 with torch.no_grad():
17     pred = model(x)
18     final_loss = loss_fn(pred, y)
19     print("loss:", final_loss.item())
20     print("weight:", model.weight.item())
21     print("bias:", model.bias.item())
22 torch.save(model.state_dict(), "model.pt")
```

运行：

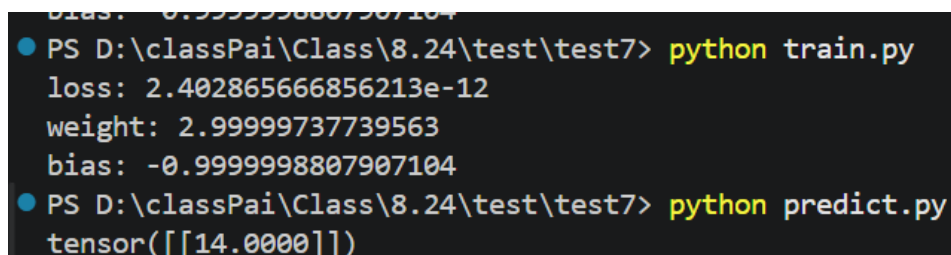
```
python train.py
```

创建文件：

```
1 #predict.py
2 import torch
3 from torch import nn
4 model = nn.Linear(1,1)
5 model.load_state_dict(
6     torch.load("model.pt")
7 )
8 model.eval()
9 x = torch.tensor([[5.]])
10 with torch.no_grad():
11     y = model(x)
12     print(y)
```

运行:

```
python predict.py
```



```
bias: -0.9999998807907104
● PS D:\classPai\Class\8.24\test\test7> python train.py
loss: 2.402865666856213e-12
weight: 2.99999737739563
bias: -0.9999998807907104
● PS D:\classPai\Class\8.24\test\test7> python predict.py
tensor([[14.0000]])
```

图 20 运行结果

3 Git 与作业提交

3.1 仓库内容整理

```
8.24/
|-- q01/
|-- q02/
|-- q03/
|-- q04/
|-- q05/
|-- q06/
|-- q07/
|-- q08/
|-- q09/
|-- q10/
|-- q11/
|-- q12/
`-- lab_report/
    |-- figures/
    |-- report.tex
```

```
|-- report.pdf
|-- report2.tex
|-- report2.pdf
|-- report3.pdf
`-- report3.tex
```

3.2 GitHub / Gitee 提交记录

仓库地址: <https://github.com/haru-senya/8.24.git>

3.3 commit 记录截图、仓库页面截图

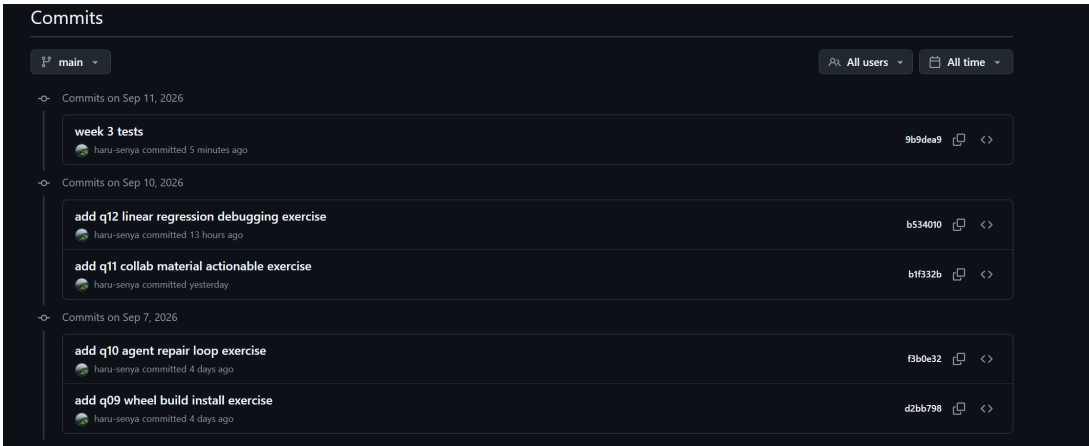


图 21 Git commit 记录

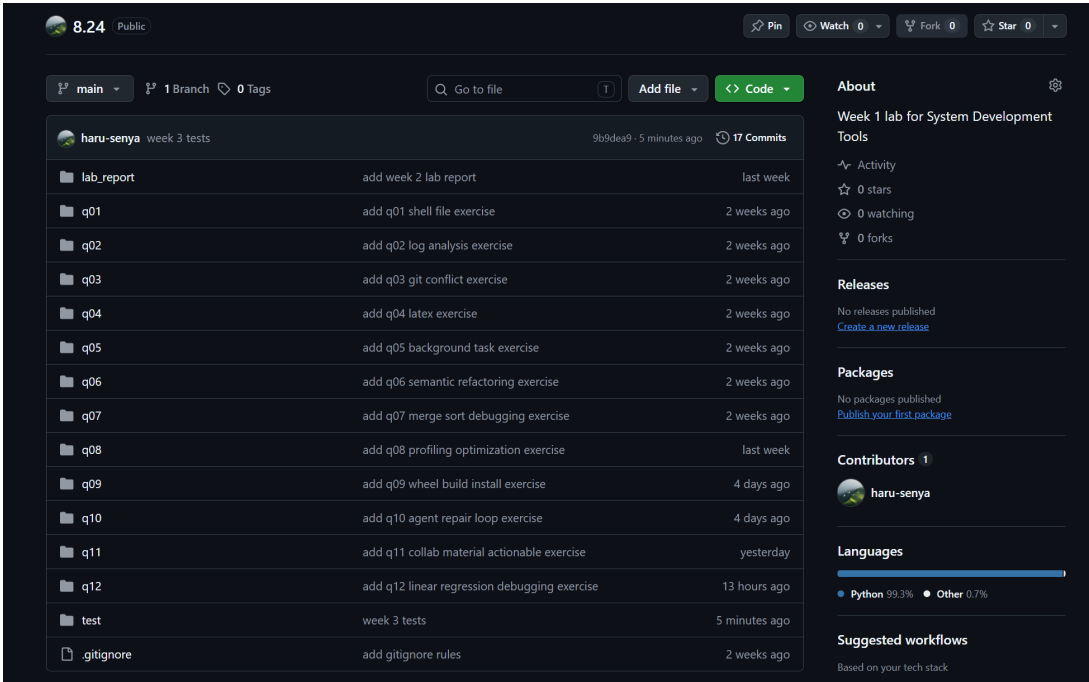


图 22 GitHub / Gitee 仓库页面

4 问题与总结

4.1 遇到的问题

- **Python 环境中缺少依赖：**在完成 q09 时运行 `python -m build` 时提示 `No module named build`。检查发现当前 Python 环境中没有安装构建工具，因此使用 `pip install build` 安装。安装过程中由于网络访问 PyPI 较慢，切换到国内镜像源后成功完成安装。
- **Python 模块导入错误：**q10 测试实验中运行 `pytest` 时出现 `ModuleNotFoundError`。检查项目结构后发现测试环境无法找到对应模块。通过调整项目目录结构和安装方式，使测试代码能够正确导入目标模块。
- **pytest 测试失败定位问题：**在 q10 中第一次运行测试时，测试未通过，提示空字符串输入情况下程序没有抛出 `SystemExit`。根据测试信息定位到参数校验逻辑缺失，修改代码后重新运行 `pytest`，最终测试通过。通过该过程理解了测试驱动开发中“发现问题—修改代码—重新验证”的流程。
- **训练结果输出位置错误：**初次运行 q12 时，将 `loss`、`weight` 和 `bias` 的输出放在训练循环内部，导致终端输出大量重复信息。调整代码结构，将最终结果输出移动到训练结束后的评估阶段，使输出符合实验要求。

4.2 实验总结

通过本次实验，我进一步熟悉了 Python 项目开发过程中的工具链使用，包括项目打包、虚拟环境管理、自动化测试以及深度学习环境配置。

在 q09 实验中，我学习了如何通过 `pyproject.toml` 配置 Python 项目，并使用 `python -m build` 生成 wheel 包，再通过独立虚拟环境完成安装验证，理解了 Python 项目从源码到可发布包的基本流程。

在 q10 实验中，我掌握了 `pytest` 测试工具的基本使用方法，并通过测试失败结果定位程序问题，理解了自动化测试在软件开发中的作用。实验过程中也认识到合理的项目结构和模块命名对于代码维护的重要性。

在 q12 实验中，我完成了基于 PyTorch 的线性回归训练流程，理解了梯度清零、反向传播和参数更新的基本过程。同时，通过配置 PyTorch 环境和调试代码错误，提高了对 Python 开发环境和深度学习工具使用的熟练程度。

总体来说，本次实验不仅提升了对 Python 工程化开发工具的使用能力，也让我认识到软件开发不仅需要实现功能，还需要关注环境管理、自动化测试、代码规范和问题定位能力。在后续学习中，需要继续加强对开发工具链和工程实践流程的掌握。